Лабораторная работа 2.
Основы программирования на Python (100 баллов).

Консольное приложение для управления списком студентов и их оценок (CSV)

Цели работы:

- Освоить базовые конструкции Python: условия и циклы.
- Научиться проектировать функции и процедуры, разбивать программу на модули и пакеты.
- Применить коллекции Python (списки, словари) для хранения и обработки данных.
- Освоить обработку ошибок и исключений, проектирование собственных исключений.
- Научиться работать с файлами: чтение/запись, формат CSV.
- Освоить базовую автоматизацию тестирования с помощью pytest.

Задачи:

1. Спроектировать пакет приложения, реализующий:
    - Хранение данных о студентах: id (int), ФИО (str), список оценок (List[int]).
    - Расчет индивидуального и группового среднего балла.
    - Добавление, удаление и обновление студентов и их оценок.
    - Сортировку и фильтрацию списка студентов.
2. Реализовать чтение/запись CSV:
    - Поддержка заголовка (header) и файлов без заголовка.
    - Корректная обработка пустых ячеек, выравнивание количества колонок оценок при записи.
3. Реализовать консольное меню (CLI) с пунктами:
    - Загрузка из CSV.
    - Сохранение в CSV.
    - Показ всех студентов.
    - Добавление студента.
    - Удаление по id.
    - Обновление оценок по id.
    - Показ статистики по группе.
    - Экспорт ТОП-N студентов в CSV по среднему баллу.
    - Сортировка по avg|name|id и показ результата.
4. Обработать типичные ошибки:
    - Некорректные данные в CSV и пользовательском вводе.
    - Отсутствие файла, нехватка прав, неверный формат.
    - Дубликаты id, отсутствие студента при удалении/обновлении.
5. Написать автотесты на pytest, покрывающие:
    - Парсинг и валидацию данных (модели, обработка).
    - Чтение/запись CSV.
    - Базовый сценарий работы CLI.

Исходные данные (формат CSV)

- Минимальные колонки: id, name; далее 0..N колонок gradeK.
- Пример строки с хедером: id,name,grade1,grade2,grade3
- Оценки — целые числа от 0 до 100. Пустая ячейка — отсутствующая оценка.
- Имена допускают пробелы и кириллицу, кодировка файлов — UTF-8.

Функциональные требования

- Расчет среднего для студента как среднее по имеющимся оценкам (пустой список -> 0.0).
- Групповая статистика: число студентов, общий средний балл по всем оценкам, лучший и худший студент по среднему баллу.
- Сортировка:
  - avg: по убыванию среднего, затем по имени (a..z) для стабильности.
  - name: по имени (a..z).
  - id: по возрастанию id.
- Экспорт ТОП-N: CSV с колонками id, name, average, grades (оценки в одной ячейке строкой через пробел).

Нефункциональные требования

- Версия Python: 3.9+.
- Кодировку файлов задавать явно (UTF-8), newline="" при работе с csv.writer/csv.reader.
- Разделить код на модули и пакет. Использовать аннотации типов, понятные имена, докстринги.
- Исключения: собственная иерархия (базовое + специализированные).
- Не использовать внешние библиотеки, кроме pytest для тестов.

Требования к структуре проекта

- Пакет приложения, например: lab/ (errors.py, models.py, io_utils.py, processing.py, main.py).
- Папка data/ с примером входного CSV.
- Папка tests/ с автотестами pytest.
- Точка входа: запуск через python -m lab.main.

Обработка ошибок и исключений

- Валидация CSV-строк: типы, пустые поля, диапазоны, недостаток колонок.
- Прерывание выполнения программы недопустимо: ошибки должны перехватываться и выводиться понятным сообщением.
- Информативные сообщения для пользователя при работе с файлами (нет доступа, файл не найден, неверный формат).

Тестирование

- Установить pytest: pip install pytest
- Запуск тестов: pytest -q
- Рекомендуется покрыть:
    - Валидные/невалидные кейсы парсинга оценок и CSV.
    - Раундтрип запись->чтение CSV.
    - Сортировку, фильтрацию, статистику.
    - Минимальный сценарий CLI с monkeypatch ввода.

Порядок выполнения работы

1. Спроектировать модель данных (класс Student: id, name, grades, свойство average).
2. Реализовать функции обработки: валидация, добавление/удаление/обновление, сортировка, статистика.
3. Реализовать ввод/вывод CSV: чтение с хедером/без, запись с хедером и выравниванием колонок оценок, экспорт ТОП-N.
4. Реализовать консольное меню (циклы, условия) с обработкой ошибок.
5. Написать автотесты pytest.
6. Самопроверка: локальный прогон приложения и тестов.
7. Подготовить отчет и материалы к сдаче.

Состав сдаваемых материалов

- Архив/репозиторий с исходным кодом: пакет приложения, tests, data (пример CSV).
- Файл README с инструкциями по запуску, описанием функций и примеров.
- Отчет (PDF/MD): цель, задачи, описание архитектуры, примеры запуска (скриншоты), результаты тестов, выводы.
- Опционально: requirements-dev.txt (pytest) и конфигурация CI.

Оценивание (100 баллов)

- Меню, условия и циклы (корректная навигация, отсутствие падений) — 15 баллов.
- Работа с коллекциями (структуры, сортировки, фильтрации, списковые включения) — 15 баллов.
- Функции и процедуры (декомпозиция, чистые функции, повторное использование) — 10 баллов.
- Модули и пакеты (структура, изоляция ответственности) — 10 баллов.
- Обработка ошибок и собственные исключения (валидация, try/except, информативные сообщения) — 10 баллов.
- Работа с файлами и CSV (чтение/запись, хедер, выравнивание, кодировка) — 15 баллов.
- Экспорт ТОП-N студентов — 5 баллов.
- Статистика группы (count, overall_avg, best, worst) — 5 баллов.
- Автотесты pytest (покрытие ключевых сценариев, устойчивость) — 10 баллов.
- Качество кода (аннотации типов, докстринги, читаемость, базовый стиль PEP 8) — 5 баллов.

Дополнительные баллы (+ до 10, сверх 100 не начисляются)

- Расширенный CLI: аргументы командной строки (например, --load, --save, --top) — до +4.
- Дополнительные форматы: экспорт JSON или Markdown — до +3.
- Дополнительная статистика: медиана/мода, распределение оценок — до +3.
- Настроенный CI (GitHub Actions) с прогоном pytest — до +3.

Штрафы

- Отсутствие обработки ошибок (падения приложения) — до -15.
- Жестко захардкоженные пути/кодировка — -5.
- Непрозрачные сообщения об ошибках — -5.
- Несоответствие структуре проекта — -5.
- Тесты не запускаются или падают из-за синтаксических ошибок — до -10.
- Опоздание со сдачей — по правилам дисциплины.

Критерии зачета

- Набрано не менее 60 баллов и работа защищена устно.
- Программа запускается, выполняет базовые сценарии без сбоев, тесты проходят.

Вопросы для защиты

1. Разница между модулем и пакетом, как Python находит модули (PYTHONPATH, **init**.py).
2. Отличия списков, кортежей и словарей; когда использовать каждый.
3. Как организована сортировка в Python, ключ сортировки и стабильность.
4. Особенности работы с CSV в Python (csv.reader/csv.writer, newline, кодировка).
5. Иерархия и назначение собственных исключений; когда вводить специализированный тип.
6. Алгоритм расчета общего среднего по группе; обработка пустых наборов.
7. Подходы к валидации пользовательского ввода и данных из файлов.
8. Что и зачем тестировать в CLI, как использовать monkeypatch/capsys в pytest.

Примечания

- Разрешено использовать готовый шаблон/реализацию, если он соответствует требованиям и вы его понимаете (должны уметь защитить каждое решение).
- Обязательно проверяйте приложение на некорректных данных (пустые строки, нечисловые оценки, пропуски).

Структура каталогов:

```
project_root/
  lab/
    __init__.py
    errors.py
    models.py
    io_utils.py
    processing.py
    main.py
  data/
    students.csv
```

Пример входного CSV:

```
id,name,grade1,grade2,grade3
1,Иванов Иван,78,85,90
2,Иванов Петр,65,70,0
3,Иванова Анна,92,88,95
4,Иванов Олег,100,90,
5,Иванова Юлия,80,,
```

Структура тестов:

```
project_root/
  tests/
    conftest.py
    test_models.py
    test_processing.py
    test_io_utils.py
    test_main_cli.py
```

Пример pytest.ini:

```
[pytest]
# Устанавливаем кодировку вывода (например, utf-8)
# В pytest нет прямой опции для кодировки, но можно установить переменные окружения
# или использовать фикстуры, если необходимо. Но для уверенности можно указать warning фильтры:

# Фильтры предупреждений (filterwarnings)
filterwarnings =
    ignore::DeprecationWarning
    ignore::PendingDeprecationWarning
    ignore::UserWarning

# Пример других опций
addopts = -ra -q  # рапорт по тестам + тихий режим
testpaths = tests  # каталог с тестами

# Если необходимо указать кодировку при запуске тестов, это задается окружением Python,
# обычно UTF-8 — это дефолт в современных средах.
```

Файл `pytest.ini` поддерживает опцию `filterwarnings`, где можно явно игнорировать или показывать предупреждения.

Пример requirements-dev.txt:

Вариант с использованием `pytest` и `flake8`:

```
pytest>=8.4.2
flake8>=6.0.0
```

Более универсальный вариант с `ruff`:

```
pytest>=8.4.2
ruff>=0.0.300  # версия примерная, лучше указать последнюю стабильную на момент установки
```